NANOBRIDGE
CONECTANDO LA FICCIÓN CON LA REALIDAD

Semana 3 · T1 — Esquema lógico y plan de pruebas (carga de prospectos)

Código	Versión	Fecha	Estado	Responsable	Revisión de Dirección
Semana 3	—	14 y 15-sep-2026	Hecho	Daniela	Por registrar

Entregable de la semana 3 del programa integrado de 16 semanas para el rol T1 (Datos y estandarización): *"Diseñar esquema de carga, staging y base maestra; pruebas de duplicados"* → **"Esquema lógico + plan de pruebas"**.

Implementa el módulo 3 de `PLAN_CRM_DEFINITIVO.md` ("Prospectos e importación") y el grupo `/prospectos` de `PLAN_API_DEFINITIVO.md`.

1. Esquema lógico

1.1 Por qué una tabla de staging y no escribir directo

Ninguna fila de CSV o alta manual toca `empresas`/`contactos`/`prospectos` directamente. Primero cae en `borradores_captura`, se valida y se deduplica ahí, y solo se promueve cuando un usuario confirma explícitamente. Esto cumple dos reglas ya cerradas en `PLAN_CRM_DEFINITIVO.md`:

- *"Los borradores viven en `borradores_captura`, expiran a los 30 días y no activan automatización."*
- *"La razón social nunca fusiona prospectos automáticamente."*

1.2 Tabla `borradores_captura`

Grupo de columnas	Contenido
Identidad del lote	`lote_id` (un archivo CSV o un alta manual = un lote), `fuente`, `fila_numero`, `fila_original` (JSON crudo, para auditar si el mapeo es cuestionable)
Datos de empresa/contacto	`empresa_*` (nombre legal, giro, tamaño, región, estado, ciudad, país, sitio web), `contacto_nombre`, `contacto_puesto` — **nullable**: una fila sin nombre de empresa o de contacto igual se registra, para no perder trazabilidad de filas incompletas
Medios de contacto	`correo`, `telefono` (+ sus versiones `_normalizado`), `canal_inicial`
Contexto comercial	`confianza`, `prioridad`, `score`, `fuente_url`, `observaciones`, `campana_id`
Estado y resultado de la revisión	`estado` (`pendiente_revision` / `duplicado` / `importado` / `rechazado` / `expirado`), `match_contacto_id`, `match_motivo`, `errores` (JSON)
Trazabilidad	`prospecto_id` (una vez promovida), `creado_por`, `creado_en`, `expira_en` (+30 días), `procesado_en`, `procesado_por`

1.3 Ciclo de vida de una fila

```
CSV o alta manual
        │
        ▼
```

```
¿pasa validación?  ──No──►  rechazado (con detalle en `errores`)
       │ Sí
       ▼
¿coincide con otra fila     ──Sí──►  duplicado (match_contacto_id = NULL,
del mismo lote?                      requiere resolver la otra fila primero)
       │ No
       ▼
¿coincide con un contacto   ──Sí──►  duplicado (match_contacto_id = X,
existente en BD?                     confirmar con usar_contacto_existente=true)
       │ No
       ▼
pendiente_revision
       │
       ├── confirmar ──► importado (crea empresa+contacto+prospecto en transacción)
       ├── rechazar  ──► rechazado
       └── 30 días sin decisión ──► expirado (job diario)
```

1.4 Reglas de validación (módulo 3 de `PLAN_CRM_DEFINITIVO.md`)

- Correo: formato válido si viene.
- Teléfono: al menos 7 caracteres si viene.
- Debe venir correo **o** teléfono (no se puede prospectar sin ningún medio).
- El canal inicial declarado debe tener su medio correspondiente en la fila (ej. `canal: correo` exige que `correo` venga lleno).

1.5 Reglas de deduplicación

Orden exacto que pide el plan — **primero correo normalizado, después teléfono normalizado** — aplicado dos veces:

1. Contra otras filas del mismo lote (evita dar de alta dos veces la misma cuenta si el archivo trae un duplicado interno).
2. Contra `medios_contacto` ya existentes en BD (mismo criterio que ya usa `automatizacion.service.registrarProspecto` para n8n, para no tener dos lógicas de dedup divergentes).

2. Plan de pruebas

#	Caso	Cómo se prueba	Resultado esperado
P1	Fila válida, sin coincidencias	CSV con datos limpios	`pendiente_revision`
P2	Correo mal formado	CSV con un correo inválido	`rechazado`, `errores` explica el campo
P3	Sin correo ni teléfono	Fila con ambos vacíos	`rechazado`
P4	Canal sin su medio correspondiente	`canal: correo` sin campo `correo`	`rechazado`
P5	Dos filas del mismo archivo con el mismo correo	CSV con una fila repetida	Primera `pendiente_revision`, segunda `duplicado` (sin `match_contacto_id`)
P6	Fila que coincide con un contacto ya en BD	Correo ya existente en `medios_contacto`	`duplicado` con `match_contacto_id`
P7	Confirmar una fila `pendiente_revision`	`POST .../confirmar`	Crea empresa+contacto+prospecto; fila pasa a `importado`
P8	Confirmar una fila `duplicado` sin `usar_contacto_existente`	`POST .../confirmar` sin el flag	`409`, no crea nada
P9	Confirmar una fila `duplicado` de otra fila del lote (sin `match_contacto_id`)	`usar_contacto_existente :true`	`409` — pide resolver la otra fila primero
P10	Un agente intenta ver/confirmar el lote de otro agente	Rol `agente`, lote ajeno	`404` (no `403`, para no revelar que el lote existe)
P11	Borrador con más de 30 días sin decisión	Job de limpieza diario	Pasa a `expirado`

#	Caso	Cómo se prueba	Resultado esperado
P12	CSV con encabezados pero cero filas de datos	Archivo vacío	`400`, mensaje explícito

2.1 Resultado obtenido con datos reales (STEELSAFE NANO® , 30 cuentas)

Se mapeó `Prospeccion_industrial_STEELSAFE_NANO.xlsx` (hoja "Prospectos") al contrato de columnas del importador y se corrió contra `parseCsv`/`filaCsvSchema` reales (no una reimplementación aparte). Cubre P1, P2, P3, P4 y P5 con datos de producción en vez de sintéticos:

- **1ª corrida:** 27 filas listas, 3 rechazadas.
  - 1 fila sin correo ni teléfono publicado (solo formulario web) — P3, esperado: coincide con el criterio de "requiere verificación humana" que ya marca la propia base de STEELSAFE.
  - 2 filas con correo inválido — resultó ser P2 disfrazado: el Excel original traía **más de un correo en la misma celda**, separados por `"|"` (ej. `"correo1@x.com | correo2@x.com"`), lo que rompe el formato de un solo correo. Se corrigió el mapeo tomando el primero.
- **2ª corrida (tras el ajuste): 29 de 30 filas listas para confirmar**, 0 duplicadas dentro del propio archivo, 1 rechazada (la de solo formulario web, correcta).

2.2 Pendiente de validar (requiere MySQL real, no disponible en el entorno donde se construyó esto)

- P6, P7, P8, P9, P10, P11 requieren una base de datos real corriendo — se validaron por revisión de código y por analogía con el mismo patrón ya probado end-to-end en el resto del backend (commit `ff2225d`, Fabián — race conditions, transacciones), pero no con una corrida real todavía.
- Siguiente paso: `npm run migrate` contra MySQL local y subir `steelsafe_prospectos_import.csv` de verdad al endpoint `POST /api/v1/prospectos/importaciones`, confirmar algunas filas y verificar en BD que empresa/contacto/prospecto quedaron bien ligados.

3. Estado

Campo	Valor
Entregable	Esquema lógico + plan de pruebas (T1, semana 3)
Fecha	14 de septiembre de 2026
Evidencia de código	`015_prospectos_importacion.sql`, `schema.ts`, `shared/csv.ts` (`parseCsv`), `crm/prospectos.service.ts`, `crm/prospectos.controller.ts`
Evidencia de prueba	§2.1 de este documento (29/30 filas reales de STEELSAFE validadas)
Pendiente	§2.2 — validación end-to-end contra MySQL real